# Comparing Random Forests, MLPs and CNNs for PathMNIST Image Classification

Group 64
Student 1 — SID: 540935875
Student 2 — SID: 550087894
Student 3 — SID: 520271713
Student 4 — SID: 510469531

COMP5318/COMP4318 — Semester 2, 2026

## 1 Introduction

This study compares a random forest (RF), a multilayer perceptron (MLP), and a convolutional neural network (CNN) on nine-class classification of small colour pathology images. We investigate how their representations and model structures affect predictive performance and computational cost, and how validation-based hyperparameter selection changes their performance. All three models are trained from scratch on the course-provided PathMNIST subset.

Microscope images contain colour and texture patterns that can support tissue classification. This experiment provides a controlled setting for examining how different algorithms use these patterns; it does not establish clinical performance. Comparing algorithms is useful because a higher classification score can come with greater training cost, while a simpler method can provide a practical baseline. The objective is therefore to explain the observed trade-offs, rather than to maximise accuracy with an unrestricted model or search budget.

## 2 Data

### 2.1 Data description and exploration

The course data contain 32,000 training images and 8,000 test images, each with dimensions $28 \times 28 \times 3$. Pixel intensities are stored as unsigned 8-bit integers in $[0, 255]$. The assignment identifies these images as a subset of PathMNIST, containing microscope views of normal and abnormal body tissues. PathMNIST is part of the MedMNIST collection described by Yang et al. [1]. We retain the supplied split rather than using the original MedMNIST split. The supplied arrays do not contain a class-name mapping, so this report uses the observed class IDs 0–8 rather than assigning unverified tissue names.

**Table 1:** Class counts in the supplied data.

| Class ID | 0 | 1 | 2 | 3 | 4 | 5 | 6 | 7 | 8 |
|---|---|---|---|---|---|---|---|---|---|
| Training | 3490 | 3431 | 3505 | 3656 | 2950 | 4290 | 2728 | 3253 | 4697 |
| Test | 873 | 858 | 877 | 914 | 737 | 1072 | 682 | 813 | 1174 |

The largest training class accounts for 14.68% of samples and the smallest for 8.53%. This is a moderate imbalance, with approximately 1.72 times as many samples in the largest class as in the smallest. Always predicting the largest training class yields a training majority-class baseline of

14.68%. We report macro F1 alongside accuracy so that smaller classes receive equal weight in one summary measure.

Training-channel means are $(188.65, 135.93, 180.05)$ and standard deviations are $(31.91, 45.60, 32.18)$ for red, green and blue. These statistics show different intensity distributions across channels and motivate retaining colour information. The small image size limits the detail available for classification. Colour similarity between images and variation in local texture are potential challenges: aggregate colour alone may not distinguish images whose spatial arrangements differ. Figure 1 supports inspection of within-class variation and between-class similarities without assuming that each class has a single visual prototype.

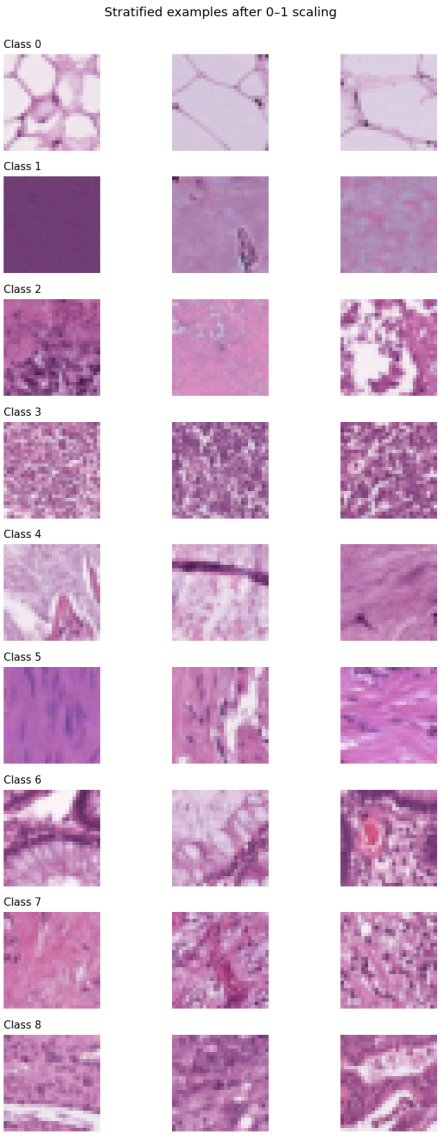



**Figure 1:** Three training examples from each class after pixel scaling. The sample selection uses a fixed random seed.

## 2.2 Pre-processing

For both neural networks, pixels are converted to float32 and divided by 255, preserving the image shape while placing inputs in $[0, 1]$. This changes numerical scale without removing colour or spatial information. Labels remain integer class IDs, matching the sparse categorical cross-entropy loss.

For RF, each image is represented by 597 deterministic features. Non-overlapping $2 \times 2$ RGB mean pooling produces $14 \times 14 \times 3 = 588$ features. Three channel-wise standard deviations describe colour variability, and six mean absolute adjacent-pixel differences describe horizontal and vertical roughness in the three channels. This reduces the input dimension from 2,352 to 597, retaining coarse location and some texture information while reducing the number of candidate feature values the trees process. The trade-off is loss of fine detail and the limited descriptive power of hand-designed summaries.

These operations are applied independently to each image and fit no statistics on validation or test samples. No augmentation, channel standardisation or further dimensionality reduction is used. Retaining the small native resolution avoids introducing interpolation choices; retaining colour avoids discarding a potentially useful signal. Augmentation could be useful, but its effect is outside this compact comparison and remains a limitation rather than an established unnecessary step.

A stratified 80/20 split of the supplied training set produces 25,600 development images and 6,400 validation images, with random seed 42. All candidates use this same split. The test labels are inspected for descriptive counts but test prediction scores do not guide parameter selection. Once settings are fixed, models are retrained on all 32,000 training images.

# 3 Methods

## 3.1 Theory

**Random forest.** RF aggregates decision trees fitted to bootstrap samples, with random feature subsets considered during splitting [2]. A tree partitions feature space through a sequence of threshold decisions; averaging tree class-probability estimates reduces dependence on an individual tree. Diversity between trees helps control variance, although correlated trees can still make similar errors. This choice connects directly to the Week 5 course topics, Decision Trees and Ensembles: a random forest is an ensemble of decision trees.

Among the methods introduced in the first six weeks, RF offers a suitable baseline for the pooled colour and texture features. Unlike a linear logistic-regression classifier (Week 3), it can represent nonlinear feature interactions without explicitly constructing interaction terms. Unlike Naïve Bayes (Week 4), it does not assume conditional independence between features, which is useful when neighbouring image regions are correlated. Compared with a single decision tree (Week 5), the ensemble aims to reduce variance and sensitivity to the training sample. These properties motivate its selection; they do not establish that it would outperform the other course methods, which were not tested here. RF also provides a useful comparison with MLP and CNN because it learns tree-based decision rules over fixed features rather than a neural representation.

**Multilayer perceptron.** An MLP transforms the flattened image through learned affine mappings and nonlinear activations. For a hidden layer, the central operation is $h = \mathrm{ReLU}(Wx + b)$, with batch normalisation inserted in this implementation. The softmax output defines probabilities over nine classes. Training minimises

$$\mathcal{L} = -\frac{1}{N} \sum_{i=1}^{N} \log p(y_i \mid x_i)$$

using Adam [3]. Dense layers can learn interactions among all pixel positions, but do not explicitly encode image locality or reuse the same detector across locations.

**Convolutional neural network.** A CNN applies shared filters across spatial positions. Local receptive fields and weight sharing are central principles of convolutional image recognition [4].

Early layers learn small patterns, while later layers combine information over larger regions. Weight sharing reduces the number of parameters needed to detect a pattern at multiple positions. Max pooling reduces spatial resolution and provides some tolerance to small changes in feature position, at the cost of detail. The resulting representation is passed to dense layers and a softmax classifier, trained using the same classification loss as the MLP.

### 3.2 Strengths and weaknesses

RF is fast in this experiment and can represent nonlinear feature interactions. Individual tree decisions are inspectable, although explaining a 120-tree ensemble is harder than explaining a single tree, and feature-importance analysis is not performed here. Its main representational limitation is reliance on fixed summaries: pooling can erase fine structures that the classifier cannot recover.

The MLP learns directly from all pixels and can model complex interactions, but its first dense layer is parameter-heavy. Its treatment of image positions as separate inputs can make learning recurring local patterns less efficient. Batch normalisation supports optimisation [5], and dropout regularises the network by randomly omitting units during training [6], but neither explicitly introduces the spatial structure available to a CNN.

The CNN has an inductive bias suited to local image patterns. Shared filters allow a compact parameterisation, but convolutions can still require many operations at every spatial position. Consequently, fewer parameters do not guarantee faster training. Pooling can discard discriminative information in already small images, and learned filters do not by themselves make the full classifier readily interpretable. These theoretical differences motivate examining both predictive scores and measured runtime.

### 3.3 Architecture and hyperparameters

The MLP uses Flatten, Dense($H$, without bias), batch normalisation, ReLU and dropout, followed by the same sequence with $H/2$ units and a nine-unit softmax layer. Omitting the two hidden-layer biases avoids redundant offsets before batch normalisation. The CNN uses two $3 \times 3$ same-padded convolution layers with $F$ and $2F$ filters, each followed by $2 \times 2$ max pooling, then Flatten, Dense(64, ReLU), dropout and a nine-unit softmax layer. Both are intentionally small to allow repeated searches.

**Table 2:** Full factorial search spaces: eight candidates per algorithm.

| Model | Hyperparameter | Values and intended effect |
|---|---|---|
| RF | Number of trees | 60, 120: more averaging, with additional fitting cost |
| | Maximum depth | 12, 20: larger values permit more complex partitions |
| | Minimum leaf size | 1, 3: larger leaves constrain fine-grained fitting |
| MLP | First hidden-layer width | 128, 256: controls capacity and parameter count |
| | Dropout | 0.10, 0.25: controls regularisation strength |
| | Initial learning rate | 0.0005, 0.001: controls optimisation step size |
| CNN | First-layer filters | 16, 24: controls the number of learned feature maps |
| | Dropout | 0.15, 0.35: trades fitting capacity against regularisation |
| | Learning rate | 0.0003, 0.001: controls optimisation step size |

A validation-set grid search provides a transparent comparison at manageable cost. Two values per parameter allow selected interactions to be examined without an extensive search; they do not establish a global optimum. Cross-validation would require substantially more neural-network fits. Candidates are ranked by validation accuracy, then macro F1 for exact accuracy ties. RF uses four worker threads. Neural networks use batches of 128, with prediction batches of 256.

MLP search runs for at most 30 epochs, with early stopping after six epochs without validation-accuracy improvement and restoration of the best weights. A validation-loss plateau of three epochs halves its learning rate, down to a floor of $10^{-5}$. CNN search runs for at most ten epochs, with early-stopping patience two and no learning-rate reduction. These different budgets make the comparison one of practical configurations rather than equal training opportunities.

Small initial models are fitted before tuning. Initial validation accuracy is 70.48% for RF; the best validation accuracy over five initial epochs is 35.41% for MLP and 66.98% for CNN. Because neural search changes training duration as well as settings, improvement over these initial runs cannot be attributed to hyperparameter changes alone.

# 4 Results and discussion

## 4.1 Hyperparameter tuning

All reported outputs and figures are taken from the saved local rerun, using Python 3.13.2, NumPy 2.4.4, scikit-learn 1.6.1 and Keras 3.15.1 with the torch backend. The local environment used CPU PyTorch 2.11.0. Timings refer to this run only.

Tables 3–5 preserve every candidate's saved validation scores and elapsed fit-plus-validation-prediction time. Neural epoch counts refer to the best validation-accuracy epoch, not necessarily the last epoch executed.

**Table 3:** RF validation search.

| Depth | Min. leaf | Trees | Accuracy (%) | Macro F1 | Time (s) |
|---|---|---|---|---|---|
| 12 | 1 | 60 | 69.41 | 0.6718 | 10.12 |
| 12 | 1 | 120 | 69.97 | 0.6768 | 19.80 |
| 12 | 3 | 60 | 69.70 | 0.6739 | 9.68 |
| 12 | 3 | 120 | 70.08 | 0.6784 | 19.36 |
| 20 | 1 | 60 | 71.64 | 0.7008 | 14.06 |
| 20 | 1 | 120 | **72.39** | **0.7078** | 26.77 |
| 20 | 3 | 60 | 72.11 | 0.7050 | 12.86 |
| 20 | 3 | 120 | 72.38 | 0.7077 | 25.90 |

Increasing RF depth from 12 to 20 improves validation accuracy for every matched tree-count and leaf-size setting. At depth 20, increasing the tree count from 60 to 120 improves accuracy by approximately 0.75 percentage points with leaf size 1 and 0.27 points with leaf size 3. At depth 12 with leaf size 1, the gain is approximately 0.56 points. More trees increase fitting time substantially, while leaf-size effects depend on depth and tree count. The best candidate uses depth 20, leaf size 1 and 120 trees, reaching 72.39% validation accuracy and .7078 macro F1.

**Table 4:** MLP validation search. $H$ is the first hidden-layer width.

| $H$ | Dropout | Initial LR | Accuracy (%) | Macro F1 | Best epoch | Time (s) |
|---|---|---|---|---|---|---|
| 128 | 0.10 | 0.0005 | 63.77 | 0.6199 | 30 | 81.54 |
| 128 | 0.10 | 0.0010 | 64.72 | 0.6338 | 30 | 81.99 |
| 256 | 0.10 | 0.0005 | 64.25 | 0.6348 | 30 | 89.33 |
| 256 | 0.10 | 0.0010 | 59.50 | 0.5720 | 23 | 85.09 |
| 128 | 0.25 | 0.0005 | 65.59 | 0.6354 | 27 | 73.05 |
| 128 | 0.25 | 0.0010 | 65.91 | 0.6413 | 29 | 74.03 |
| 256 | 0.25 | 0.0005 | **66.22** | **0.6479** | 27 | 93.44 |
| 256 | 0.25 | 0.0010 | 62.33 | 0.6120 | 30 | 89.77 |

MLP performance depends on interactions between width, dropout and optimisation. Increasing dropout from .10 to .25 improves validation accuracy in all four matched width–learning-rate comparisons in this rerun. A higher learning rate improves both 128-unit settings but reduces accuracy in both 256-unit settings. The best validation candidate uses 256 units, dropout .25 and learning rate .0005, reaching 66.22% accuracy and .6479 macro F1 at epoch 27. Its search fit takes 93.44 seconds. The 128-unit candidate with dropout .25 and learning rate .001 reaches 65.91% accuracy; it is slightly below the winning 256-unit configuration selected for final training.

**Table 5:** CNN validation search. $F$ is the first convolutional layer's filter count.

| $F$ | Dropout | LR | Accuracy (%) | Macro F1 | Best epoch | Time (s) |
|---|---|---|---|---|---|---|
| 16 | 0.15 | 0.0003 | 70.55 | 0.6995 | 10 | 58.39 |
| 16 | 0.15 | 0.0010 | 73.48 | 0.7247 | 9 | 59.80 |
| 24 | 0.15 | 0.0003 | 68.81 | 0.6831 | 9 | 86.34 |
| 24 | 0.15 | 0.0010 | **75.84** | **0.7452** | 9 | 85.46 |
| 16 | 0.35 | 0.0003 | 69.33 | 0.6799 | 10 | 60.98 |
| 16 | 0.35 | 0.0010 | 74.50 | 0.7330 | 10 | 60.49 |
| 24 | 0.35 | 0.0003 | 67.53 | 0.6600 | 10 | 89.47 |
| 24 | 0.35 | 0.0010 | 68.72 | 0.6657 | 5 | 64.13 |

CNN dropout .15 outperforms .35 in three of the four matched comparisons. The exception is 16 filters with learning rate .001, where dropout .35 reaches 74.50%, compared with 73.48% for dropout .15. At dropout .15, learning rate .001 improves accuracy for both filter counts. Increasing the filter count helps at this higher rate but reduces accuracy at the lower rate. Thus, neither greater capacity nor stronger regularisation is uniformly beneficial. Ten-epoch runs take about 58–61 seconds with 16 filters and 85–89 seconds with 24 filters. The 24-filter, dropout-.35, learning-rate-.001 candidate peaks at epoch 5 and takes 64.13 seconds; its shorter elapsed time reflects early stopping.

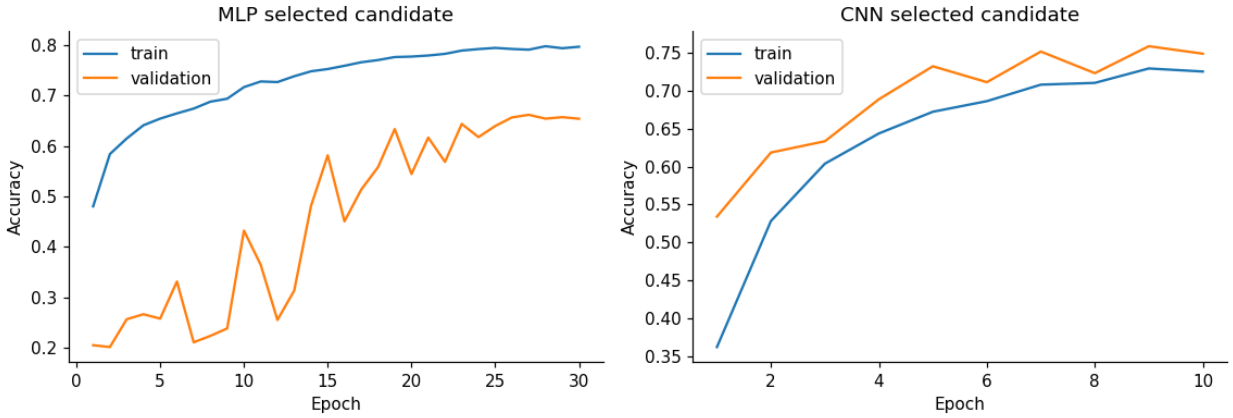



**Figure 2:** Training and validation accuracy for the selected MLP and CNN search candidates. These curves describe development-set training, not the final full-training-set fits.

The best MLP search candidate reaches its highest validation accuracy at epoch 27 of 30; the CNN reaches its best at epoch 9 of 10. Several candidates peak near the epoch cap, limiting claims of complete optimisation. Figure 2 shows the best search candidates, including the 256-unit MLP. These curves concern development-set fitting, while final models are retrained on all supplied training images. Training and validation curves are not identical-condition estimates: dropout is active during training but disabled for validation, and training metrics are accumulated while weights change within an epoch. A validation score above a training score is therefore not necessarily anomalous.

### 4.2 Final model comparison

Final configurations are frozen from the saved validation search and written explicitly in separate notebook cells, so final training does not require rerunning the search. RF uses 120 trees, depth 20 and minimum leaf size 1. MLP uses hidden widths 256/128, dropout .25 and initial learning rate .0005 for 27 epochs, matching the best MLP candidate and its best validation epoch. The retained learning-rate history shows that reduced rates first apply at epochs 10, 14, 19, 23 and 26; this schedule is fixed for full-data training without consulting held-out labels. CNN uses 24/48 filters, dropout .15 and learning rate .001 for nine epochs, also matching its search winner. Each final model is trained on all 32,000 training images and evaluated on the 8,000-image test set. Transferring the validation-selected schedule does not guarantee an identical optimisation trajectory on the larger training set.

**Table 6:** Final settings and held-out test results. Timing excludes preprocessing and hyperparameter search.

| | RF | MLP | CNN |
|---|---|---|---|
| Actual final settings | 120 trees; depth 20; leaf 1 | 256/128 units; dropout .25; LR .0005; 27 epochs | 24/48 filters; dropout .15; LR .001; 9 epochs |
| Accuracy (%) | 72.9375 | 66.5625 | **76.5875** |
| Macro F1 | .7147 | .6557 | **.7572** |
| Weighted F1 | .7227 | .6631 | **.7622** |
| Training time (s) | 34.9975 | 97.6540 | 88.8766 |
| Prediction time, 8,000 images (s) | .1424 | .3566 | 1.1339 |
| Input features / model parameters | 597 input features | 637,577 parameters | 162,265 parameters |

CNN exceeds RF accuracy by 3.65 percentage points and MLP accuracy by 10.03 points. Its macro F1 advantage indicates better average class-wise F1 as well as better aggregate accuracy. The result is consistent with the usefulness of learned local representations, although architecture and representation vary together and neither effect is isolated causally.

RF is a strong computational baseline: its final training takes about 39% of the CNN time and exceeds MLP accuracy by 6.38 percentage points. This supports the usefulness of coarse colour and texture summaries on this subset. Even with its validation-selected configuration, MLP does not outperform RF. Its larger dense representation and longer training budget do not translate into higher test accuracy in this experiment.

CNN has approximately 75% fewer counted parameters than MLP and takes about 91% of its total training time, while using nine epochs rather than 27. Convolutional filters are reused across spatial locations, so parameter count alone does not determine arithmetic cost; the different epoch budgets also affect total runtime. RF prediction is fastest in the saved results. Timings are single-fit wall-clock observations in the local CPU environment, without uncertainty estimates, and should not be compared directly with GPU timings from another environment. RF feature count and neural parameter counts describe different quantities rather than a common model-size measure.

**Table 7:** Selected class-wise test recalls (%).

| Model | Class 0 | Class 1 | Class 2 | Class 3 | Class 6 | Class 7 |
|---|---|---|---|---|---|---|
| RF | 96.56 | 93.01 | 43.10 | 91.47 | 43.11 | 54.24 |
| MLP | 95.42 | 90.44 | 41.85 | 84.03 | 32.70 | 60.27 |
| CNN | 94.04 | 93.24 | 43.56 | 92.89 | 60.56 | 60.39 |

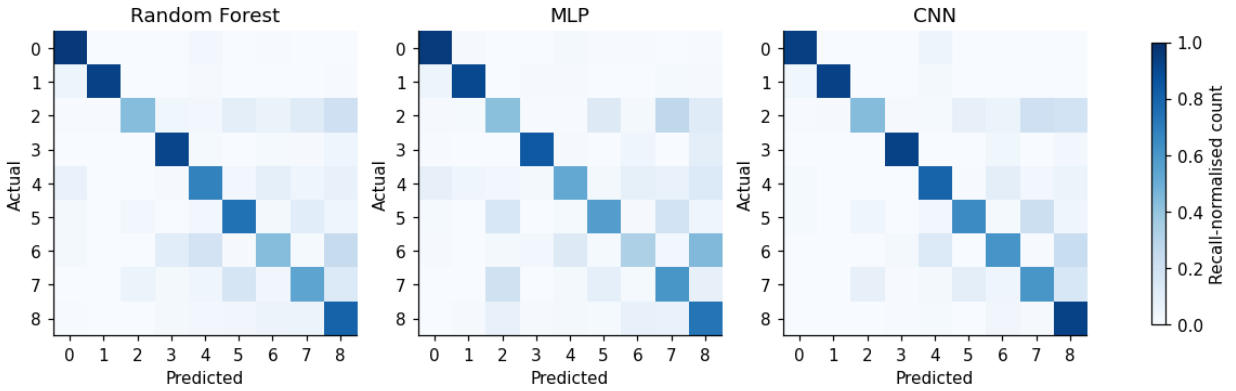



**Figure 3:** Row-normalised test confusion matrices. Rows are actual classes and columns are predictions; diagonal entries are class recall.

All three models have relatively high recall for classes 0 and 1, whereas classes 2 and 6 are more difficult. CNN improves class-6 recall from RF's 43.11% to 60.56%, while MLP recall for this class is only 32.70%. CNN class-2 recall remains low at 43.56%. CNN is not best on every measure: RF has higher class-0 recall. CNN class-7 precision is 51.85% despite recall of 60.39%, indicating many false positive assignments to this class. These differences show why a single overall accuracy is insufficient. Confusion patterns identify areas for investigation but do not establish a biological explanation.

## 5 Conclusion

Within this course subset and search budget, CNN provides the strongest predictive performance, achieving 76.59% test accuracy and .7572 macro F1. RF offers a useful speed–performance compromise, while the tested MLP is both less accurate and slower to train than RF. The CNN result is consistent with the benefit of local feature learning and shared weights, but costs more training time than RF, while taking slightly less total training time than the selected MLP. No interpretability or clinical-validation experiment is conducted, so conclusions on those aspects remain limited.

The main limitations are a single validation split and random seed, small search grids, different neural training budgets, a transferred learning-rate schedule, and no uncertainty estimates. Architecture and input representation also vary together. The supplied image-level split provides no evidence here about patient-level independence or transfer to other acquisition settings. Several classes remain difficult despite the aggregate CNN improvement.

Future work should repeat the complete validation-based selection over several seeds to assess stability, without using test scores to choose settings. Increasing neural epoch budgets with a consistent stopping policy would test whether near-cap best epochs reflect incomplete training. An RF feature ablation comparing pooled colour alone with the added roughness summaries would quantify their contribution. Finally, modest rotation/flip augmentation, after checking that these transformations preserve labels, could be evaluated on validation data to test robustness to tissue orientation. Each extension addresses a specific uncertainty in the present experiment.

# 6 Reflection

### Student 1 — SID: 540935875

The main lesson I draw from selecting a classical baseline is that an algorithm should be chosen by matching its assumptions to the data. Week 3 logistic regression offers a straightforward linear classifier, while Week 4 Naïve Bayes uses a conditional-independence assumption. Neither property automatically makes these methods unsuitable, but correlated colour and texture features give a reason to consider the nonlinear decision rules of Week 5 trees. Random forest extends this idea by combining diverse trees rather than relying on one partition of the training data. This comparison helps me distinguish a reasoned design choice from a claim of superior performance: our results support RF as a useful baseline, but cannot show that it is better than logistic regression, Naïve Bayes or SVM because those alternatives were not evaluated. In a future study, I would test one of these alternatives under the same validation protocol to examine the selection argument empirically.

### Student 2 — SID: 550087894

The most important lesson I take from the data-processing part of this assignment is that preprocessing determines which information a model can use. Scaling pixels to $[0, 1]$ changes their numerical range while preserving the image, whereas the pooling used for RF removes fine spatial detail. This distinction becomes clear when comparing the neural networks' image inputs with the forest's 597 colour and roughness features. The forest still achieves 72.94% test accuracy, showing that a compact representation can be useful, but this result alone does not prove that pooling improves accuracy. The task also highlights why sample images and class counts should be inspected before training: they provide context for deciding what information to preserve and how to interpret performance. In future work, I would compare feature sets under the same validation conditions rather than judge a preprocessing step only by whether it reduces input size.

### Student 3 — SID: 520271713

The main lesson I take from tuning the neural networks is that a final accuracy score needs to be interpreted together with the training process. In the MLP search, increasing dropout improves all four matched configurations, whereas increasing learning rate helps the 128-unit settings but harms the 256-unit settings. This makes it clear that a setting cannot be labelled helpful without considering the rest of the model. The learning curves add another useful insight: the best MLP search candidate peaks at epoch 27, and several candidates reach their best score close to the epoch limit. The selected settings and learning-rate schedule are recorded explicitly so that final training can run independently of the expensive search. A weaker result may therefore reflect the optimisation schedule or training budget as well as the architecture. I would now examine learning curves, the best epoch and elapsed time before drawing conclusions from a search table. For a future comparison, I would use a more consistent training-budget policy and repeat runs to check whether the apparent improvements are reliable.

### Student 4 — SID: 510469531

The most important lesson I take from analysing the final results is that the best overall score does not describe every aspect of a model's usefulness. CNN achieves 76.59% test accuracy, yet its class-2 recall is only 43.56%. Its class-7 precision is also only 51.85%, showing that identifying many examples of a class does not necessarily mean that predictions of that class are reliable. Reading

these scores alongside the confusion matrices gives a more informative picture than ranking models by accuracy alone. Runtime changes the comparison again: RF is less accurate overall but takes substantially less time to train. This task therefore helps me see evaluation as an explanation of trade-offs and failure patterns, rather than simply the selection of a winner. Future improvements should target a clearly identified weakness and be assessed on validation data, without using the final test results to repeatedly adjust the model.

## 7 AI Acknowledgement

Our learning was grounded in the weekly tutorial code and course concepts. We used OpenAI Codex to help explain and correct code, assist with model implementation and experiment checks, and summarise and interpret selected results. Codex also helped draft and polish the report and format it in LaTeX. The reported numerical results come from the executed notebook.

## References

[1] J. Yang, R. Shi, D. Wei, Z. Liu, L. Zhao, B. Ke, H. Pfister, and B. Ni, "MedMNIST v2 – A large-scale lightweight benchmark for 2D and 3D biomedical image classification," *Scientific Data*, vol. 10, article 41, 2023. doi: 10.1038/s41597-022-01721-8.

[2] L. Breiman, "Random Forests," *Machine Learning*, vol. 45, pp. 5–32, 2001. doi: 10.1023/A:1010933404324.

[3] D. P. Kingma and J. Ba, "Adam: A Method for Stochastic Optimization," arXiv preprint arXiv:1412.6980, 2014. `https://arxiv.org/abs/1412.6980`.

[4] Y. LeCun, L. Bottou, Y. Bengio, and P. Haffner, "Gradient-based learning applied to document recognition," *Proceedings of the IEEE*, vol. 86, no. 11, pp. 2278–2324, 1998. `https://bottou.org/papers/lecun-98h`.

[5] S. Ioffe and C. Szegedy, "Batch Normalization: Accelerating Deep Network Training by Reducing Internal Covariate Shift," in *Proceedings of the 32nd International Conference on Machine Learning*, PMLR, vol. 37, pp. 448–456, 2015. `https://proceedings.mlr.press/v37/ioffe15.html`.

[6] N. Srivastava, G. Hinton, A. Krizhevsky, I. Sutskever, and R. Salakhutdinov, "Dropout: A Simple Way to Prevent Neural Networks from Overfitting," *Journal of Machine Learning Research*, vol. 15, no. 56, pp. 1929–1958, 2014. `https://jmlr.org/papers/v15/srivastava14a.html`.